# Node.js — Day 21

Environment Variables ■

**Long-term learning notes**
These notes capture the concepts, distinctions, examples, questions, and practical discoveries from Day 21 so they can be used later for revision and backend development.

## 1. What is an Environment Variable?

An **environment variable** is a value stored outside application source code that a running application can access. It is commonly used for configuration and sensitive values.

Common examples:

• **PORT** — the port on which an application runs
• **DATABASE_URL** — database connection configuration
• **API_KEY** — an API credential
• **NODE_ENV** — the application's environment, such as development or production

**Core idea:** Keep configuration outside your source code so the same application code can run with different configuration values in different environments.

## 2. Why Do We Use Environment Variables?

Instead of hardcoding configuration or secrets directly in JavaScript:

```
const apiKey = "my-secret-key"; // ■ hardcoded
```

we can provide the value through the environment and let the application read it.

**Important:** Environment variables are not automatically secure. Their benefit is that secrets/configuration do not have to be written directly into source code or committed to a repository.

## 3. What is process.env?

**process.env** is provided by Node.js and gives the running Node.js process access to environment variables that are available to it.

Example:

```
console.log(process.env.PORT);
```

If the process has PORT available as 3000, the value accessed through **process.env.PORT** is generally the string **"3000"**.

## 4. Environment Variable Values Are Strings

Environment variable values are generally received as strings. This matters when the application expects another data type, such as a number.

```
const port = Number(process.env.PORT);
```

If PORT is "3000", Number(...) converts it to the number 3000.

> **Remember:** process.env.PORT → typically a string
> Number(process.env.PORT) → number

## 5. What is a .env File?

A **.env** file is a common and convenient place to store application-specific environment configuration.

```
.env
```

```
PORT=3000
NODE_ENV=development
API_KEY=your-secret-value
```

**Important distinction:** .env and process.env are not the same thing.

Simply creating a .env file does not mean every Node.js process will automatically read its contents. The values must be loaded/provided to the running process.

## 6. How .env Connects to process.env

The mental model we established:

**.env**
↓
load/provide the variables
↓
**Node.js process environment**
↓
**process.env**
↓
your JavaScript application

> **Key distinction:** process.env does not mean “the .env file.” It means the environment variables available to the running Node.js process.

## 7. Loading a .env File with Node.js

Modern Node.js provides an environment-file option that can be used when starting a program:

```
node --env-file=.env readEnv.js
```

This tells Node.js to read the variables from **.env**, make them available in the Node.js process environment, and then run **readEnv.js**.

> **Very important:** Node.js is not executing the .env file like a JavaScript file. It is reading the variables from the file and making them available to the process.

## 8. Practical Experiment — What Happened?

You tested the concept yourself. Before loading the .env file, your code produced values such as:

```
undefined
NaN
undefined
```

This happened because process.env.PORT and process.env.NODE_ENV were not available to that Node.js process.

Then you ran:

```
node --env-file=.env readEnv.js
```

and the values became available.

This demonstrated:

.env exists → variables are stored there
↓
Node is started with --env-file=.env
↓
variables become available to the process
↓
readEnv.js can access process.env

## 9. Why Did NaN Appear?

When PORT was unavailable, process.env.PORT was **undefined**. Therefore:

```
Number(undefined)
```

produced:

```
NaN
```

**NaN** means “Not a Number.” The important lesson is that converting an unavailable environment variable does not magically produce a valid number.

## 10. .gitignore and Secrets

If .env contains private configuration or secrets, it is commonly added to **.gitignore** so it is not accidentally committed to Git.

```
.gitignore
```

```
.env
```

> **Security reminder:** .env is not a secure vault. Anyone who obtains the file can see its contents. Its common purpose is to keep configuration/secrets out of source code and repositories.

## 11. Accept vs Environment Variables — Not the Same Topic

Do not confuse HTTP headers such as **Accept** and **Content-Type** from Day 19 with environment variables. HTTP headers travel with HTTP requests/responses, while environment variables provide configuration to a running application.

| Concept | Purpose |
|---|---|
| HTTP header | Metadata about an HTTP request/response |
| Environment variable | Configuration/value available to a running application |
| .env | Common file used to store application environment configuration |
| process.env | Node.js access to environment variables available to the process |

## 12. Practical Example

.env:

```
PORT=4000
NODE_ENV=production
```

readEnv.js:

```
const port = Number(process.env.PORT);
const environment = process.env.NODE_ENV;

console.log(port);
console.log(environment);
```

Run:

```
node --env-file=.env readEnv.js
```

Expected output:

```
4000
production
```

## 13. Mini Challenge — Completed ■

You correctly wrote:

```
const port = Number(process.env.PORT);
const environment = process.env.NODE_ENV;

console.log(port);
console.log(environment);
```

This demonstrated that you can read an environment variable, convert a numeric value, and access another configuration value.

## 14. Your Important Clarification

You asked whether node --env-file=.env readEnv.js means Node.js runs the .env file. The correct mental model is:

> **--env-file=.env** tells Node.js to read the variables from .env and load them into the Node.js process environment **before** running the specified JavaScript file.

Therefore, when **readEnv.js** runs, it can access those loaded values through **process.env**.

## ■ Long-Term Cheat Sheet

| Term | Remember |
|---|---|
| .env | A common file for storing application environment configuration |
| process.env | Node.js access to environment variables available to the running process |
| --env-file=.env | Tell Node.js to load variables from .env before running the JS file |
| PORT=3000 | The environment variable value is generally read as the string "3000" |
| Number(process.env.PORT) | Convert the PORT string to a number |
| .gitignore | Keep .env out of the Git repository |

## ■ Day 21 Completion

> **DAY 21 — COMPLETE ■**
>
> Next planned topic: **Practical process usage ■■**
>
> The goal remains understanding the Node.js foundation properly before moving to Express.